ROUTEX

Smart Query Router Using CrewAI

CrewAI · Groq LLM · FastAPI · Firebase · LangSmith · Vercel

PROJECT REPORT	Multi-Agent AI System
SUBMITTED BY	Anushay Ayat
SUBMITTED TO	Ma'am Mehwish Alam
PROGRAM	AI and Data Science, SMIT
DATE	November 2026

Contents

01 Abstract
02 Introduction
03 Objectives and Requirements
04 System Architecture
05 Technology Stack
06 Agent Design
07 Routing Algorithm
08 Backend API
09 Advanced Features
10 Frontend and User Interface
11 Firebase, LangSmith and Deployment
12 Testing and Evaluation
13 Security, Limitations and Future Work
14 Conclusion and References

01 Abstract

Large language models can answer almost any question, but a single general-purpose prompt rarely gives the best answer for every kind of request. A maths problem, a coding task and a translation each benefit from a different role, tone and set of instructions. This report presents **ROUTEX**, a Smart Query Router built with **CrewAI** and the **Groq** large language model. ROUTEX first classifies the intent of a user's query with a dedicated Router agent, then forwards the query to the specialist agent best suited to answer it.

The assignment asked for a router plus a Math agent and an Explanation agent. ROUTEX implements these and extends the system to six specialist agents (Math, Teacher, Code, Translator, Summarizer, Writer), agent pipelines, conversation memory, a batch Evaluation Lab, and a full web application with a live routing visualiser, analytics dashboard, Firebase authentication and history, LangSmith tracing and Vercel/Firebase deployment configuration.

6	5	6/6	30/36
specialist agents	REST endpoints	unit tests passing	fallback accuracy on labelled set

02 Introduction

2.1 Problem statement

When every user query is sent to one generic agent, the response quality depends entirely on a single prompt that must handle all tasks. Multi-agent designs solve this by separating concerns: a lightweight decision-maker decides *who* should answer, and focused experts decide *how*. This pattern is called **query routing**.

2.2 Motivation

- **Quality:** each agent has a role, goal and backstory tailored to one task type.

- **Transparency:** the routing decision, confidence and reasoning are visible to the user.
- **Extensibility:** adding a new capability means adding one agent and one category, not rewriting the system.
- **Robustness:** a rule-based fallback keeps the application usable when the model output is invalid.

2.3 Scope

The project covers the complete path from user interface to model call: a browser client, a FastAPI backend, the CrewAI agent layer, persistence of user history, observability, and deployment configuration. Items that were deliberately left out are listed in Section 13.

03 Objectives and Requirements

The assignment objective is to build a multi-agent AI system using CrewAI and a Groq LLM that automatically identifies a user's query and routes it to the appropriate agent. The table maps each requirement to its implementation.

Requirement	Implementation in ROUTEX
Router agent that classifies the query	`Query Router` agent; returns JSON with category, confidence and reasoning (`router_service.classify`)
Math agent	`Math Expert` agent (category MATH)
Explanation agent	`Teacher` agent (category EXPLAIN)
CrewAI framework	Agents, Tasks and Crews built with `crewai` (`agents.py` , `router_service.py`)
Groq LLM	Model configured through `GROQ_MODEL` (default `groq/openai/gpt-oss-120b`)
User enters a query and receives an answer	Web console calling `POST /api/route`
Extension: more agents	Code Assistant, Translator, Summarizer, Writer
Extension: professional interface	Single-page app with routing map, analytics, themes, command palette
Extension: observability and persistence	LangSmith tracing; Firebase Auth and Firestore history

04 System Architecture

4.1 High-level design

ROUTEX follows a four-hop pipeline: the client sends the query to the FastAPI service; the service runs the Router agent; the chosen specialist agent produces the answer; the response, together with timing for every step, returns to the client, which stores it in the user's history.

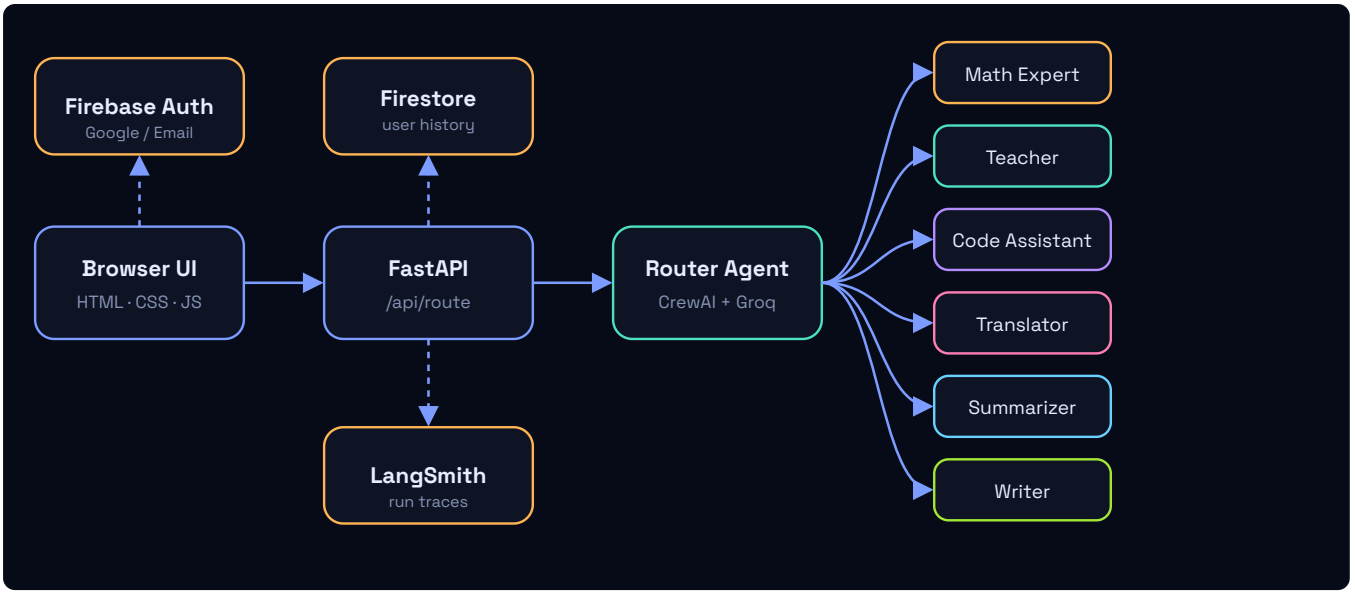


Figure 1: ROUTEX system architecture

4.2 Request flow

1. The user types a query (optionally choosing a manual agent, a pipeline, or turning memory on).
2. The browser calls `POST /api/route` with the query and options.
3. The Router agent classifies the query and returns `{category, confidence, reasoning}`. If the output cannot be parsed or validated, the rule-based fallback classifier is used.
4. The specialist agent for that category runs as a single-task CrewAI crew and returns the answer.
5. The API returns the answer, the agent metadata and a step-by-step trace with timings in milliseconds.
6. The UI animates the routing map, shows the decision panel and saves the record to Firestore (or browser storage in demo mode).

4.3 Project structure

```
smart-query-router/
├── api/index.py              Vercel serverless entrypoint
├── backend/app/
│   ├── main.py               FastAPI routes + CORS + static hosting (local)
│   ├── router_service.py     classify → specialist pipeline, chaining, memory
│   ├── agents.py             CrewAI agents + metadata
│   ├── heuristics.py         rule-based fallback classifier
│   ├── tracing.py            LangSmith helper
│   └── config.py, schemas.py
├── frontend/
│   ├── index.html
│   ├── css/  base · layout · components
│   └── js/   main · chat · viz · analytics · lab · auth · store · firebase · api · config · ui
├── tests/                    unit tests + labelled evaluation set
├── vercel.json  firebase.json  firestore.rules  render.yaml
└── requirements.txt  .env.example  README.md
```

05 Technology Stack

Layer	Technology	Purpose
Agent framework	CrewAI	Agents, tasks and crews; role/goal/backstory prompting
LLM provider	Groq (default model gpt-oss-120b)	Fast inference for routing and answers
Backend	Python, FastAPI, Pydantic, Uvicorn	REST API, validation, CORS
Frontend	HTML, CSS, vanilla JavaScript (ES modules), SVG	Interface, routing visualiser, charts
Auth and data	Firebase Authentication, Cloud Firestore	Google/email login; per-user history
Observability	LangSmith	Traces for router and specialist runs
Hosting	Vercel (serverless), Firebase Hosting, Render (backup)	Deployment targets
Testing	pytest, Playwright (screenshots)	Unit tests, evaluation script, UI capture

Plain JavaScript was chosen for the frontend so the application has no build step and can be deployed as static files to Vercel or Firebase Hosting. Charts (donut, bar, trend line) and the routing graph are drawn directly with SVG, avoiding external chart libraries.

06 Agent Design

Each agent is defined by a **role**, a **goal** and a **backstory**, the three fields CrewAI uses to build the agent's system prompt. All agents share one Groq-backed `LLM` object with a low temperature (0.2) for consistent behaviour, and delegation is disabled so that the router decides routing and agents do not hand work to each other unpredictably.

6.1 Router agent

Role: Query Router. **Goal:** classify every user query into exactly one category (MATH, EXPLAIN, CODE, TRANSLATE, SUMMARIZE, WRITE). **Backstory:** a fast, reliable dispatcher that never answers the query itself.

6.2 Specialist agents

Category	Role	Goal	Backstory
MATH	Math Expert	Solve mathematical problems accurately, showing clear steps and a final answer.	You are a precise mathematician. You verify every calculation before answering.
EXPLAIN	Teacher	Explain concepts simply with analogies and short examples.	You are a patient teacher who makes hard ideas easy for beginners.
CODE	Code Assistant	Write, debug and explain clean, correct code with brief comments.	You are a senior software engineer who writes production-quality code.
TRANSLATE	Translator	Translate text faithfully, preserving tone and meaning.	You are a professional multilingual translator.
SUMMARIZE	Summarizer	Condense text into a short, accurate summary with key points.	You are an editor who captures the essence of any text in few words.
WRITE	Writer	Draft clear, polished emails, essays, posts and stories in the requested tone.	You are a professional writer and editor who adapts tone and structure to the audience.

07 Routing Algorithm

7.1 LLM classification

The Router agent receives a task that lists the six categories with a one-line definition each, the user's query, and a strict output format: a single JSON object with `category`, `confidence` (0 to 1) and `reasoning`. The response is parsed with a regular expression that extracts the first JSON object, then validated: the category must be one of the known keys, and confidence is clamped to the range 0 to 1.

```
def classify(query: str, llm) → dict:
    agent = build_router_agent(llm)
    desc = (
        "Classify the user query into exactly ONE category from: "
        + ", ".join(CATEGORIES)
        + ".\n- MATH: calculations, equations, numeric problems\n- EXPLAIN: concepts, how/why questions\n"
        "- CODE: write/debug/explain code\n- TRANSLATE: translate text between languages\n"
        "- SUMMARIZE: shorten or summarise given text\n"
        "- WRITE: draft emails, essays, stories, posts or other writing\n\n"
        f"User query: {query}\n\n"
        'Reply with ONLY JSON: {"category": "<CATEGORY>", "confidence": <0..1>, "reasoning": "<one short
sentence>"}'
    )
    raw = _kickoff(agent, desc, "A single JSON object with category, confidence and reasoning.")
    data = _parse_json(raw)
    cat = str(data.get("category", "")).upper().strip()
    if cat not in CATEGORIES:
        raise ValueError(f"unknown category {cat}")
    conf = max(0.0, min(1.0, float(data.get("confidence", 0.8))))
    return {"category": cat, "confidence": conf, "reasoning": str(data.get("reasoning", "")).strip()}
```

7.2 Rule-based fallback

If the LLM call fails, or its output cannot be parsed or validated, the system does not fail. A deterministic classifier based on regular expressions takes over, and the trace marks the step as *fallback*. Priority order is TRANSLATE, SUMMARIZE, WRITE, CODE, MATH, with EXPLAIN as the default.

```
def classify_heuristic(query: str) → dict:
    q = query.strip()
    if _TRANS.search(q):
        cat, why = "TRANSLATE", "Detected translation keywords."
    elif _SUMM.search(q):
        cat, why = "SUMMARIZE", "Detected summarization keywords."
    elif _WRITE.search(q):
        cat, why = "WRITE", "Detected a writing or drafting request."
    elif _CODE.search(q):
        cat, why = "CODE", "Detected programming keywords."
    elif _MATH.search(q):
        cat, why = "MATH", "Detected arithmetic or math keywords."
    else:
        cat, why = "EXPLAIN", "No specific pattern matched; defaulting to explanation."
    return {"category": cat, "confidence": 0.6, "reasoning": why + " (rule-based fallback)"}
```

7.3 Specialist execution and tracing

After classification, the selected specialist runs as a one-agent, one-task sequential crew. The service measures each step with a high-resolution timer and returns a `steps` array, for example *Classify query: 640 ms*, *Generate answer: 1,420 ms*. The functions that perform classification, answering and the overall request are decorated with LangSmith's `traceable`, producing nested traces.

08 Backend API

Method and path	Purpose	Notes
`GET /api/health`	Service status	Reports model name, whether the Groq key is set, whether LangSmith tracing is on
`GET /api/agents`	Agent catalogue	Router metadata and the six specialist agents
`POST /api/classify`	Instant intent hint	Rule-based only, no LLM call; powers the live "likely → agent" hint while typing
`POST /api/route`	Route and answer	Body: `query`, optional `force_agent`, `chain`, `memory`. Max 2,000 characters
`POST /api/batch`	Classify up to 10 queries	Router only (no answers); used by the Evaluation Lab

Response format of `/api/route`

```
{
  "id": "a1b2c3d4e5f6", "query": "...", "category": "MATH",
  "confidence": 0.94, "reasoning": "Arithmetic expression detected.",
  "agent": {"key": "MATH", "role": "Math Expert", "icon": "∑", "color": "#ffb454"},
  "answer": "...", "steps": [{"step": "Classify query", "agent": "Query Router", "status": "ok", "ms": 640},
...],
  "total_ms": 2060, "model": "groq/openai/gpt-oss-120b",
  "used_fallback": false, "forced": false, "chain": []
}
```

Input validation returns HTTP 400 for an empty query, 413 when the query or batch is too large, and 500 with a clear message when the server has no `GROQ_API_KEY`.

---

09 Advanced Features

9.1 Agent chaining (pipelines)

A pipeline sends one query through two or three agents in order, and the output of each agent becomes the input of the next. For example *Summarize → Translate* first condenses a paragraph and then translates the summary. Routing is skipped for pipelines because the user has chosen the sequence; every step appears in the trace with its own timing. If one step fails, the chain stops and returns the error text.

```
def _run_chain(query: str, chain: list, llm, context: str):
    """Agent chaining: output of each agent becomes the input of the next one."""
    steps, text = [], query
    for i, cat in enumerate(chain):
        s = time.perf_counter()
        prompt = text if i == 0 else f"Apply your role to the following text and return only the
result:\n\n{text}"
        try:
            text, status = answer(cat, prompt, llm, context if i == 0 else ""), "ok"
        except Exception as exc:
            text, status = f"⚠️ The {AGENT_META[cat]['role']} could not finish: {exc}", "error"
        steps.append(dict(step=f"Pipeline step {i + 1}", agent=AGENT_META[cat]["role"], status=status,
ms=int((time.perf_counter() - s) * 1000)))
        if status == "error":
            break
    return text, steps
```

9.2 Conversation memory

When the Memory toggle is on, the client sends the last four question and answer pairs. The server truncates each turn (300 characters for the question, 400 for the answer) and prepends them to the specialist's task as context, so follow-up requests such as "make it shorter" can be understood.

9.3 Manual mode

The mode selector lets the user bypass the router and talk to a chosen agent directly (`force_agent`). The trace then marks the routing step as *skipped* with confidence 100%.

9.4 Evaluation Lab

The Lab accepts up to ten lines in the form `query | EXPECTED`, calls `/api/batch`, and presents accuracy, average confidence, number of fallbacks, a per-query table and a confusion matrix. It makes the quality of the router measurable by the user rather than asserted.

```
def classify_with_fallback(query: str, llm) → dict:
    """Classification only (no specialist answer). Used by the Evaluation Lab."""
    s = time.perf_counter()
    try:
        r, fb = classify(query, llm), False
    except Exception:
        r, fb = classify_heuristic(query), True
    return {**r, "query": query, "used_fallback": fb, "ms": int((time.perf_counter() - s) * 1000)}
```

10 Frontend and User Interface

About the screenshots. Figures in this section were captured from the running interface in demonstration mode, where responses are sample data, so that the interface can be shown without exposing API keys. The layout, animations and interactions are those of the real application.

10.1 Landing page

The hero shows a live routing demo: example queries travel from the Query node through the Router to one of six agent nodes, with a caption naming the chosen agent. A scrolling strip lists the technologies used.

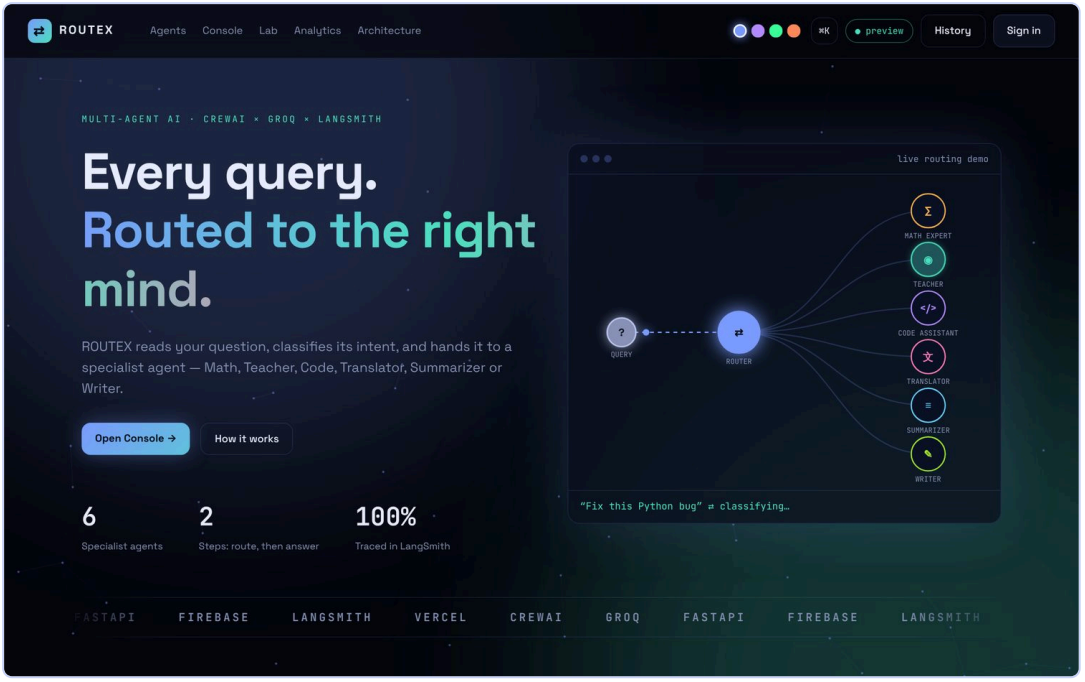


Figure 2: Landing page with the live routing demo

10.2 Agent catalogue

Six cards describe the agents. Clicking a card loads a matching example query into the console.

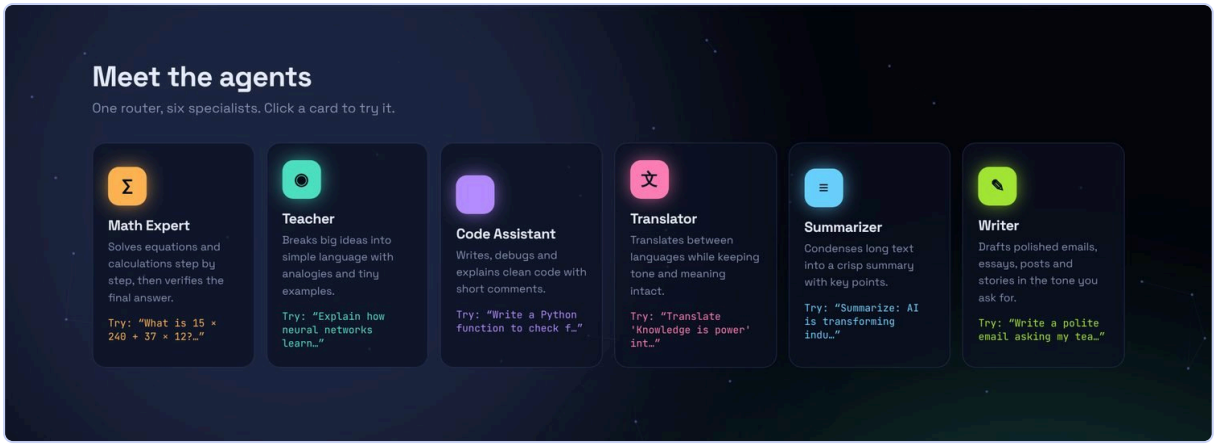


Figure 3: Meet the agents section

10.3 Routing console

The console has two panels. On the left, a chat with a mode selector, pipeline selector, memory toggle, voice input and typewriter-style answers, each with copy, text-to-speech and feedback buttons. On the right, the **Live Routing Map** animates a signal from the query through the router to the chosen agent, followed by the decision card (category, confidence meter, reasoning) and a trace of each step with its time.

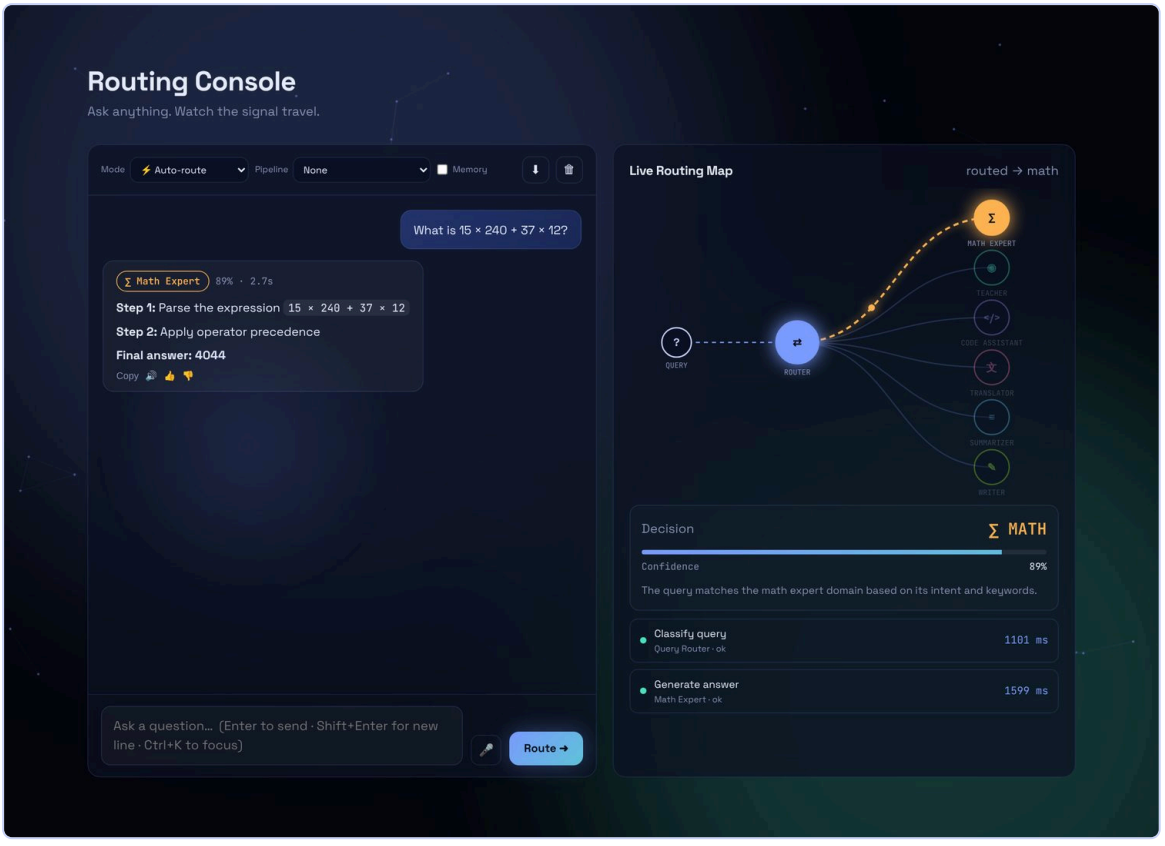


Figure 4: Console: a maths query routed to the Math Expert

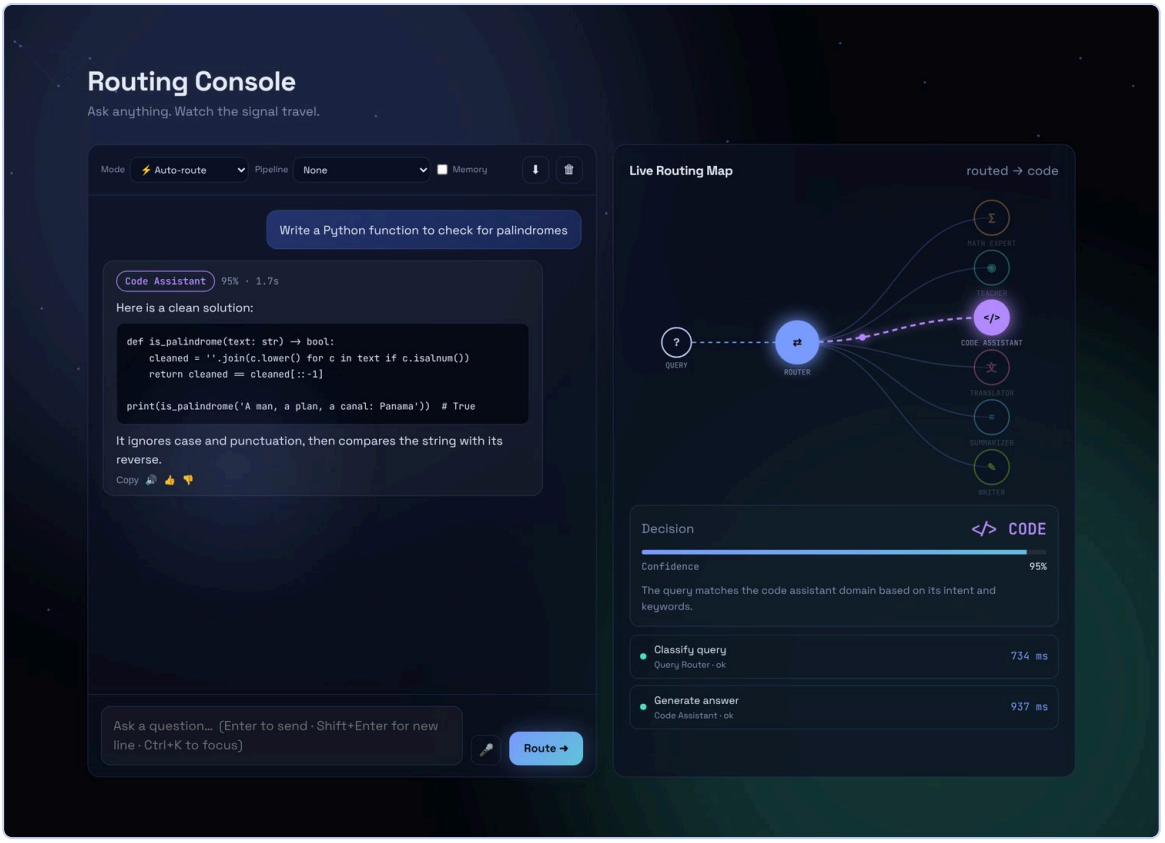


Figure 5: Console: a coding request routed to the Code Assistant

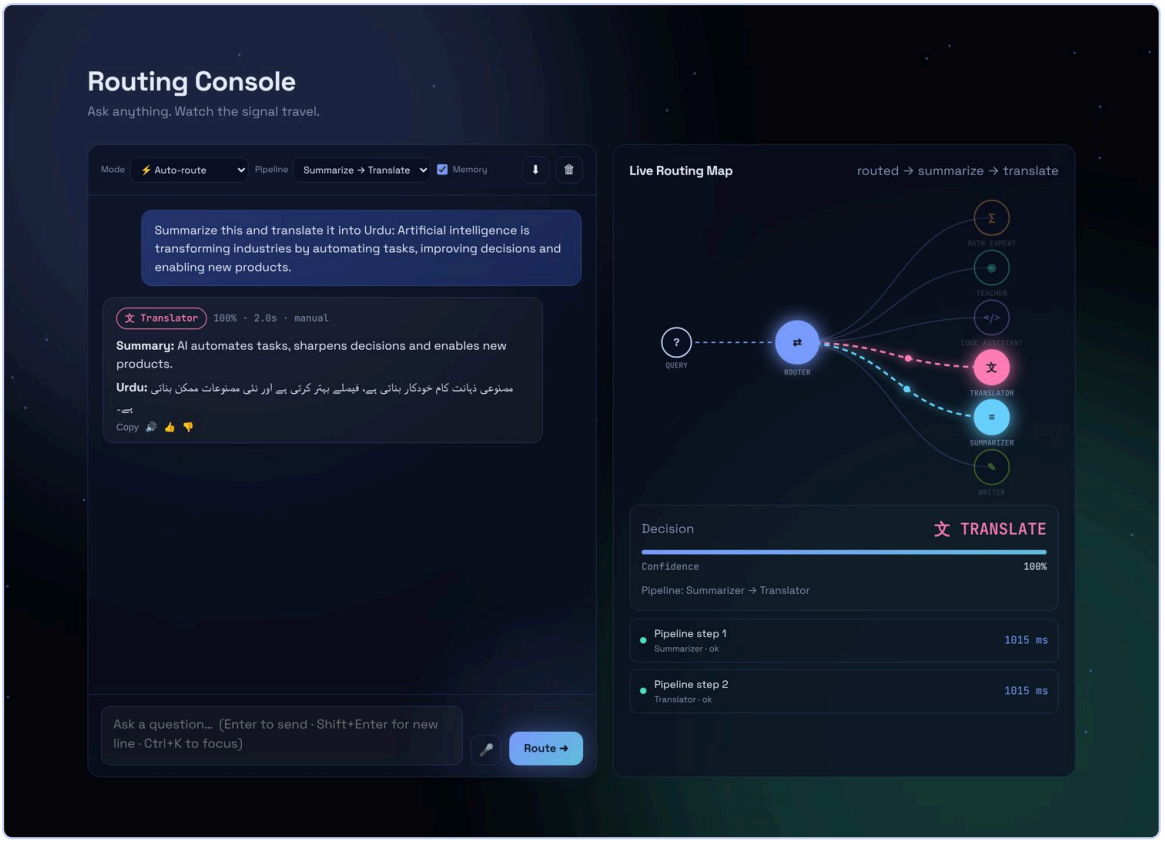


Figure 6: Console: Summarize → Translate pipeline (two agents highlighted, two trace steps)

10.4 Evaluation Lab and analytics

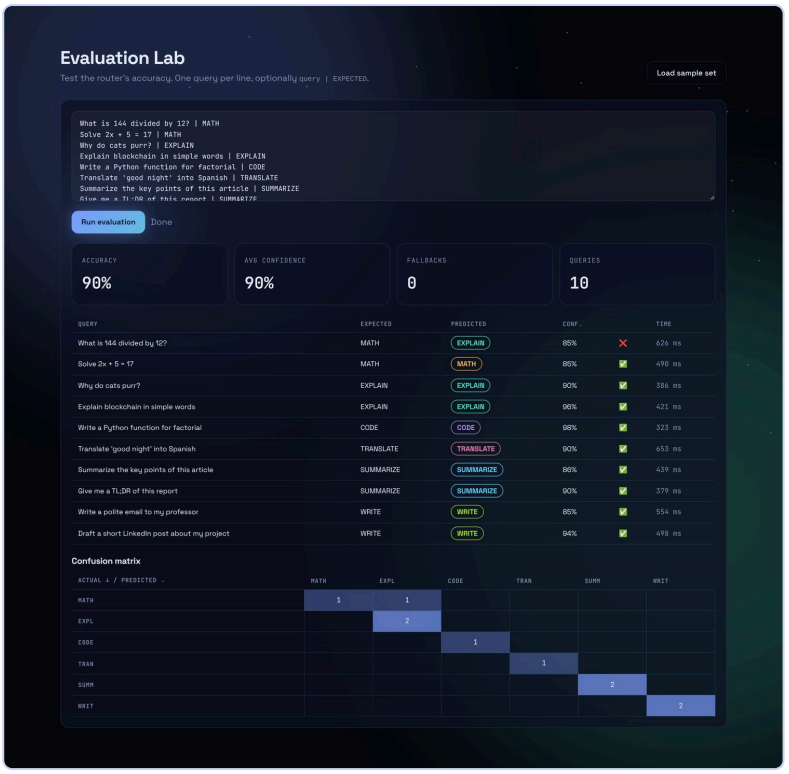


Figure 7: Evaluation Lab: per-query results, KPIs and confusion matrix (demonstration data)

The analytics dashboard is computed from the user's saved history: total queries, average latency, average confidence, most used agent, satisfaction from 👍 / 👎 feedback, an agent distribution donut, a latency bar chart and a confidence trend line.



Figure 8: Analytics dashboard (sample data loaded)

10.5 Productivity and personalisation

A command palette (Ctrl+K) jumps to any section, runs example queries, clears the chat, opens history, or changes theme. History can be searched and exported as CSV or JSON. Four themes change the accent colours of the whole interface, including the animated background.

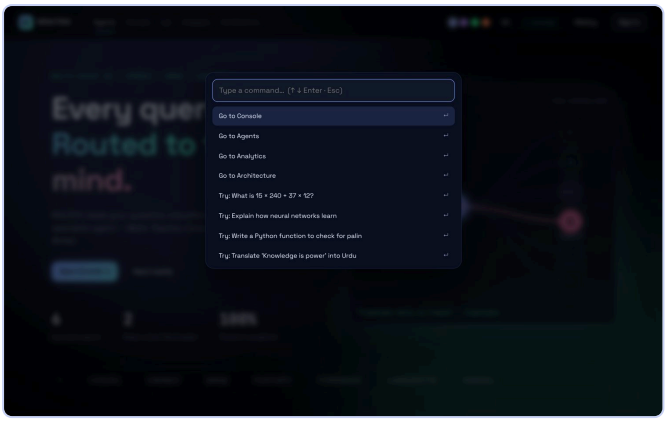


Figure 9: Command palette

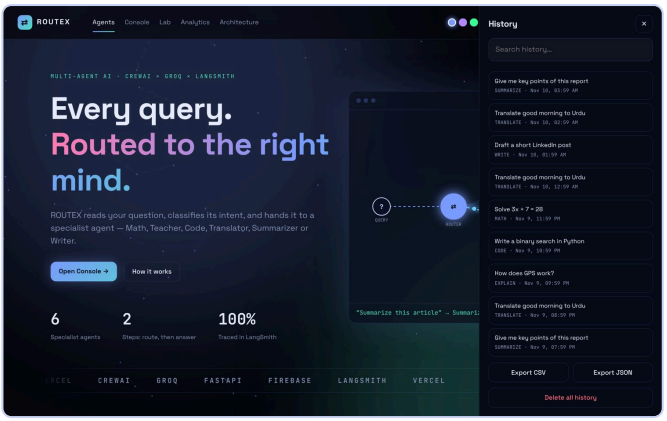


Figure 10: History drawer

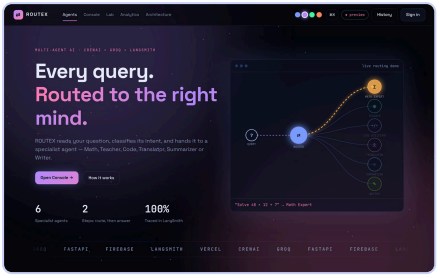


Figure 11: Purple theme

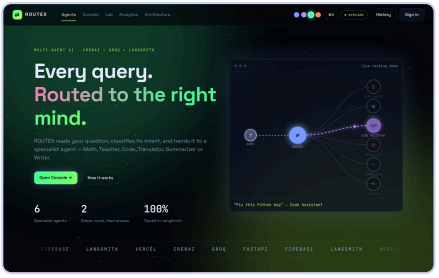


Figure 12: Matrix theme

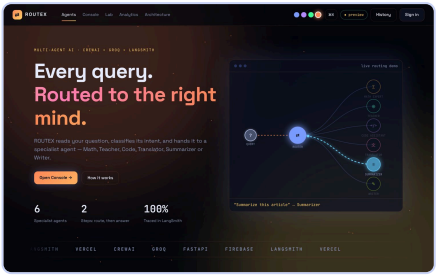


Figure 13: Sunset theme

10.6 Interface engineering notes

- No framework or build step; the code is split into small ES modules (`chat` , `viz` , `analytics` , `lab` , `auth` , `store` , `firebase` , `api` , `ui`).
- Answers are rendered through an HTML-escaping markdown renderer, so model output cannot inject markup.
- Responsive layout down to phone width; reduced-motion preference disables animations.
- Text-to-speech and voice input use the browser's Web Speech APIs where available.

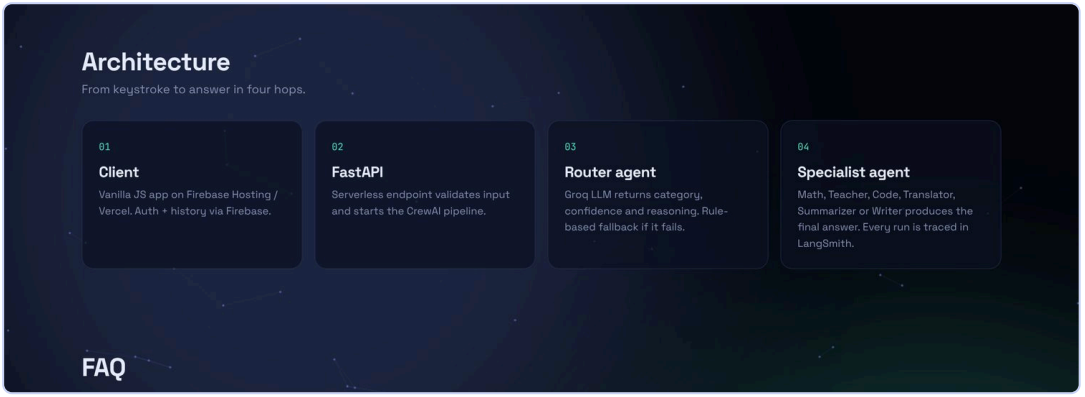


Figure 14: Architecture section of the interface

11 Firebase, LangSmith and Deployment

11.1 Firebase authentication and history

Users can sign in with Google or email and password. After login, every response is saved to Cloud Firestore under `users/<uid>/queries/<id>`. Without a Firebase configuration the application runs in demo mode and stores history in the browser. Firestore rules restrict each user to their own documents:

```
match /users/{uid}/queries/{doc} {
  allow read, write: if request.auth ≠ null && request.auth.uid == uid;
}
```

11.2 LangSmith tracing

Setting `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY` and `LANGSMITH_PROJECT` enables tracing without code changes. Each request appears as a `smart_query_router` trace with child runs `router_agent_classify` and `specialist_agent_answer` (or `router_batch_item` for the Lab), so the router's decision and each agent's output can be inspected and compared.

11.3 Deployment

Target	Contents	How
Vercel	Static frontend (`frontend/`) and serverless FastAPI (`api/index.py`)	`vercel.json` sets the output directory, a 60 s function limit and rewrites `/api/*` ; environment variables set in the dashboard
Firebase Hosting	Static frontend only	`firebase deploy --only firestore:rules,hosting` ; set `API_BASE` in `config.js` to the backend URL
Render (backup)	FastAPI backend	`render.yaml` ; used if CrewAI exceeds the Vercel function size limit

Deployment note. CrewAI is a large dependency. If Vercel rejects the function for size, the backend can run on Render with the same code and the frontend keeps working by pointing `API_BASE` to it.

12 Testing and Evaluation

12.1 Unit tests

Six pytest tests cover the rule-based classifier (maths, translation, summarisation, default explanation, writing, and a check that a coding request containing the word "write" is still classified as CODE). All six pass.

12.2 Evaluation of the rule-based fallback

A labelled set of 36 queries (six per category, including deliberately ambiguous phrasings) is stored in `tests/eval_set.json`. The script `tests/eval_fallback.py` runs the fallback classifier on it without an API key.

30/36	83.3%	6
correct	accuracy	misclassified

Category	Correct	Total	Accuracy
MATH	3	5	60%
EXPLAIN	6	7	86%
CODE	5	6	83%
TRANSLATE	6	6	100%
SUMMARIZE	4	6	67%
WRITE	6	6	100%

Actual \ Predicted	MATH	EXPLA	CODE	TRANS	SUMMA	WRITE
MATH	3	2				
EXPLAIN	1	6				
CODE		1	5			
TRANSLATE				6		
SUMMARIZE		2			4	
WRITE						6

Misclassified queries

Query	Expected	Predicted by fallback
What is 144 divided by 12?	MATH	EXPLAIN
Find the square root of 529	MATH	EXPLAIN
How do I calculate compound interest?	EXPLAIN	MATH
Create an HTML page with a login form	CODE	EXPLAIN
Condense this essay into 3 bullets	SUMMARIZE	EXPLAIN
Can you boil this long email down?	SUMMARIZE	EXPLAIN

The errors show the limit of keyword rules: phrasings such as “divided by”, “square root”, “condense” and “boil down” carry intent without a keyword the rules know, and “how do I calculate…” is a request for explanation but contains a maths keyword. These are the cases a language-model router is expected to handle better, and the fallback exists so that routing still succeeds when the model is unavailable.

12.3 Evaluating the LLM router

With a Groq key configured, the Evaluation Lab runs the same labelled queries through the Router agent and reports accuracy, confidence and the confusion matrix, which can then be compared with the fallback table above. The accuracy of the language-model router depends on the model and prompt in use and should be measured on the target deployment.

13 Security, Limitations and Future Work

13.1 Security and privacy

- The Groq and LangSmith keys live only in server environment variables; the browser never sees them.
- Inputs are validated: empty queries rejected, queries limited to 2,000 characters, batches limited to ten items.
- CORS origins are configurable through `ALLOWED_ORIGINS`.
- Firestore rules isolate each user's history; answers are rendered with HTML escaping.

13.2 Limitations

- No rate limiting or admin dashboard yet, so a public deployment should add them.
- Memory sends recent turns from the browser; there is no server-side session store.
- Answers are returned in one response rather than streamed token by token.
- The interface is English only and dark-themed.
- Heavy dependencies may require a non-serverless host for the backend.

13.3 Future work

- File and PDF upload with retrieval-augmented answers (RAG).
- Admin panel with usage statistics, API keys and rate limiting.
- Urdu interface and a light theme.
- Streaming responses and caching of repeated queries.
- Automatic multi-step plans, where the router proposes a pipeline itself.

14 Conclusion and References

ROUTEX demonstrates the query-routing pattern end to end: a small, explicit decision up front, specialist agents behind it, visible reasoning for the user, a safe fallback, and measurement tools to judge routing quality. It meets the assignment requirements (router, math agent, explanation agent, CrewAI, Groq) and extends them with four more agents, pipelines, memory, authentication, history, analytics, tracing and deployment configuration. The same structure can be reused for other domains, such as customer support or study assistants, by changing the categories and agents.

References

1. CrewAI documentation — docs.crewai.com
2. Groq documentation — console.groq.com/docs
3. FastAPI documentation — fastapi.tiangolo.com
4. Firebase documentation (Authentication, Firestore, Hosting) — firebase.google.com/docs
5. LangSmith documentation — docs.smith.langchain.com

6. Vercel documentation — vercel.com/docs

Appendix: running the project locally

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env        # add GROQ_API_KEY and LANGSMITH_API_KEY
uvicorn backend.app.main:app --reload
# open http://127.0.0.1:8000
pytest                         # unit tests
python tests/eval_fallback.py  # fallback evaluation
```